

Tapwright

L0–L3 core platform — an agentic loop-engineering approach

How we build the hardware-abstraction, bus/measurement, diagnostics and CI-runner layers of Tapwright in 26 weeks, using AI agents running in verifiable loops as the primary implementation mechanism — and where that method deliberately stops.

37

WORK LOOPS

26

WEEKS · M1–M6

6

VERIFICATION TIERS

3–4

ENGINEERS

Scope. L0 hardware abstraction, L1 bus & measurement core, L2 diagnostics engine, L3 test authoring & CI runner — all Apache-2.0, all in SKIFIN-IT-SERVICES/tapwright. **Out of scope:** L4 security testing and L5 compliance (separate private repository, per ADR-003/007), owned hardware, ODX authoring, FlexRay, and any Rust rewrite.

0. How To Read This Document

This is an execution plan, not a research document. It assumes the conclusions of the research corpus (01- through 07-) and the architecture decisions in 07-[architecture/architecture.md](#) as settled inputs, and answers one question: **how do we build L0–L3 of Tapwright in 26 weeks, using AI agents running in verifiable loops as the primary implementation mechanism?**

Section	Answers
§0.1 Repository state	What already exists in the repo, and what this plan must not duplicate
§0.2 Relationship to PROCESS.md	How this plan and the repo's existing TDD loop fit together
§1 Scope & Constraints	What we are building, what we are not, and what is non-negotiable
§2 The Agentic Model	What a "loop" is here, and why this codebase suits the method
§3 Verification Ladder	How a loop knows it is done, without a human watching
§4 Loop Substrate	The repo, harness, and CI that make loops possible — and the gaps
§5 Loop Backlog	All 37 work units, with oracles, exit criteria, and current status
§6 Schedule	M1–M6 across 26 weeks
§7 Team & Roles	Who does what when agents write most of the code
§8 Metrics	How we know the method is working
§9 Risks	Including the failure modes specific to agentic development
§10 Definition of Done	v0.1 release criteria
§11 Decisions	All eight open decisions, now closed

0.1 Current repository state (verified 2026-08-14)

The repository is **already scaffolded well beyond what a from-zero plan would assume**. This revision reflects what is actually there. Anything marked ● must not be rebuilt.

Area	Status	Detail
Licence & legal	● Done	LICENSE (Apache-2.0), NOTICE , SECURITY.md
Governance	● Done	CONTRIBUTING.md (DCO already adopted and documented), CODE_OF_CONDUCT.md , issue + PR templates, dependabot.yml
Packaging	● Done	pyproject.toml — name <code>tapwright</code> , <code>requires-python >=3.10</code> , <code>setuptools/src</code> layout, dependencies deliberately empty pending per-layer licence verification
Module skeleton	● Done	<code>src/tapwright/{hal,buses,dbc_arxml,diag,report,runner,trace}/</code> — exactly the FW-REQ-060 layout this plan specifies
Lint/format/type config	● Done	<code>ruff</code> (line-length 100; E,F,I,UP,B), <code>mypy</code> (py310), <code>.editorconfig</code> , <code>.pre-commit-config.yaml</code> with Conventional Commits enforcement
CI	● Partial	.github/workflows/ci.yml — lint + test jobs, Python 3.10/3.11/3.12 matrix, <code>pytest --cov=tapwright</code> . Carries an explicit TODO(M1) for vcan bring-up
Development process	● Done	PROCESS.md — a 5-step issue-first/test-first loop, with <code>.claude/skills/</code> implementing each step (<code>file-issue</code> , <code>test-plan</code> , <code>tdd-develop</code> , <code>checkin</code> , <code>root-cause-analysis</code>)
Requirements docs	● Copied in	<code>docs/</code> holds architecture.md , framework-requirements.md , input-variables.md , tooling-requirements.md , phase-1-requirements.md

Area	Status	Detail
Tests	● Stub only	<code>tests/test_smoke.py</code> . No <code>unit/</code> , <code>integration/</code> , <code>differential/</code> , or <code>property/</code> tiers
Fixture corpus	● Missing	No <code>fixtures/</code> directory. This is the oracle library (§4.3)
Virtual ECU	● Missing	No <code>tools/virtual_ecu/</code> . This is the highest-leverage missing piece (§4.4)
<code>AGENTS.md</code>	● Missing	No agent contract at repo root (§4.2)
Differential/property testing	● Missing	Neither the test tiers nor the CI jobs exist
Fixture-immutability guardrail	● Missing	No CODEOWNERS, no guardrail CI job (§4.5) — the primary defence against oracle capture

One correction to carry into the repo: `docs/framework-requirements.md` in the repository is a copy of the pre-correction version, so it still records `python-can` as BSD-2-Clause and `can-isotp` as LGPL-3.0. **Both are wrong** — verified 2026-08-01, `python-can` is **LGPL-3.0** and `can-isotp` is **MIT**. The corrected table, plus the new `FW-REQ-019` isolation boundary, must be synced into `docs/` (see loop INF-03).

0.2 Relationship to the repo's existing `PROCESS.md`

`PROCESS.md` already defines a development loop: `file issue` → `test plan` → `TDD (red/green/refactor)` → `checkin` → `root-cause analysis on escapes`. This plan does **not** replace it. The two operate at different altitudes and compose cleanly:

(per change)	This plan (per programme)
How one issue becomes a merged PR	What 37 units of work exist, and in what order
Step 1–2: issue + test plan	The loop SPEC (§2.1) — same artifact, with one addition: the <i>oracle</i> must be named explicitly
Step 3: <code>tdd-develop</code> red/green/refactor	The inner loop (§2.3) — this is exactly it
Step 4: checkin + one reviewer	Verify + human gate (§2.3), with one addition: the reviewer must not have supervised that loop
Step 5: root-cause analysis	Escape handling (§8) — escapes are the key quality metric

What this plan adds that `PROCESS.md` does not currently have — and these are the substantive gaps to close, not stylistic differences:

- 1. **The oracle as a named, independently-authored artifact.** A test plan written by the same agent that implements the change is not an oracle; it is a restatement of the implementation's intent. §2.1 and §4.3.
- 2. **The verification ladder (T0–T5).** Current CI is lint + pytest. There is no differential testing (T3) and no property/fuzz testing (T4), and those are the tiers that catch a *silently wrong decode* — the highest-consequence bug class in this product. §3.
- 3. **Fixture immutability.** Nothing currently stops an agent from making a failing test pass by editing the expected output. This is the single most likely way agentic development fails here. §4.5, §9.1.
- 4. **Blast-radius constraints** per work unit. §2.1.
- 5. **The backlog and schedule** — 37 loops mapped to M1–M6. §5, §6.

Where the two documents describe the same thing, `PROCESS.md` is the operational authority and this plan defers to it. Adopting this plan should produce **edits to `PROCESS.md`**, not a competing process document.

1. Scope & Constraints

1.1 In scope

Layer	Module	Deliverable
L0 Hardware abstraction	<code>tapwright.hal</code>	Vendor-neutral bus interface over SocketCAN, gs_usb, PEAK, Kvaser, Vector XL
L1 Bus + measurement core	<code>tapwright.buses</code> , <code>.dbc_arxml</code> , <code>.trace</code>	CAN/CAN-FD/LIN restbus, DBC/ARXML/LDF/A2L decode, BLF/ASC/MDF4 trace I/O
L2 Diagnostics engine	<code>tapwright.diag</code>	UDS (ISO 14229), DoIP (ISO 13400), ISO-TP, ODX read-only import, SOVD client
L3 Test authoring & CI runner	<code>tapwright.runner</code> , <code>.report</code>	pytest-native fixtures, YAML test definitions, HTML/JSON reports, CLI, container

All Apache-2.0, all in the public repository.

1.2 Explicitly out of scope

- **L4 (security testing) and L5 (compliance/orchestration).** Per [ADR-003](#) and [ADR-007](#), these live in a **separate private repository** and are not touched by any loop in this plan. The only concession here is **DIAG-05**, which builds the interception hooks a future L4 will need — retrofitting them later means rewriting L2.
- **Owned hardware** — deferred to a possible Phase 4.
- **ODX authoring** — read-only import only.
- **FlexRay** — not in v0.1.
- **Rust core** — explicitly deferred per [FW-REQ-003](#).

1.3 Non-negotiable constraints

ID	Constraint	Source / repo reality
C-1	Python 3.10+ core; no Rust rewrite this phase	FW-REQ-001/002/003 ; <code>pyproject.toml</code> <code>requires-python >=3.10</code> , CI matrix 3.10–3.12
C-2	One engine — laptop, bench, and CI container are invocation modes, never separate builds	ADR-001
C-3	Apache-2.0 across L0–L3; SPDX headers on every file	FW-REQ-050 ; LICENSE + NOTICE present
C-4	Reuse, do not rewrite: <code>python-can</code> , <code>cantools</code> , <code>udsoncan</code> , <code>doipclient</code> , <code>asammdf</code> , <code>pytest</code>	FW-REQ-010–016
C-5	<code>vcn</code> zero-hardware path is first-class, never a fallback	ADR-005
C-6	L2 public API must remain externally wrappable without a fork	ADR-004 ; already called out in PROCESS.md step 3
C-7	No copyleft dependency bundled into the distributed core without isolation	NFR-006 , FW-REQ-017/019
C-8	Full internal test suite runs on <code>vcn</code> with zero physical hardware	FW-REQ-031
C-9	LGPL isolation. <code>python-can</code> (LGPL-3.0) and <code>asammdf</code> (LGPL-3.0) are dependencies only — never vendored, forked, or frozen into a binary. FW-REQ-022 (static binary) is BLOCKED pending legal opinion	FW-REQ-019 ; DECISIONS-RECORD.md §5
C-10	No security-bypass capability. UDS <code>0x27</code> request/response mechanics only — no seed-to-key derivation, no key databases, no brute-force helpers. Enforced by CI scan (DIAG-08), not policy alone	DECISIONS-RECORD.md ; SECURITY.md

ID	Constraint	Source / repo reality
C-11	Conventional Commits + DCO sign-off on every commit	PROCESS.md §4; .pre-commit-config.yaml; CONTRIBUTING.md

1.4 Competitive benchmarks the plan is calibrated against

Derived from the vendor scrapes in [market-apps-documentation/](#). This is what "good enough to be taken seriously" means per layer — not aspiration, but the floor set by tools engineers already use.

Layer	What the field already does	Our v0.1 bar	Honest gap at v0.1
L0	TSMaster and Vehicle Spy both support multi-vendor hardware; TSMaster works with Kvaser/PEAK as well as its own	≥3 vendors through one unchanged API, vcan included	None — parity is achievable
L1	CANoe, TSMaster, Vehicle Spy all do CAN/LIN/FlexRay/Ethernet + full calibration (CCP/XCP with write)	CAN/CAN-FD/LIN/Eth restbus, DBC/ARXML/LDF/A2L read	No FlexRay, no calibration write. Deliberate. State it plainly in docs
L2	Vehicle Spy does full ODX read and write ; CANoe has CANDelaStudio/ODXStudio authoring	UDS + DoIP + ODX read-only + SOVD client	No ODX authoring. Deliberate; SOVD partly compensates (§ below)
L3	CANoe has CI via Jenkins (vendor-documented) and a public demo repo; ecu.test ships a named CI/CD platform (one:cx); RemotiveLabs is CI-native with Volvo/Ford as customers	pytest-native, git-diffable, one engine across laptop/bench/CI	CI-native is no longer an empty lane. Our edge is open-core + no dongle + no per-environment SKU, not "we have CI"

Two findings that trim our claimed advantages — worth internalising before writing marketing copy:

- 1. Vector runs a browser-based CANoe Online Demo** (support KB0023426). Our "try it with zero install" pitch is therefore weaker than assumed. The durable version of the claim is "*no install and fully scriptable and free and it runs in your own pipeline*" — not "no install."
- 2. CI-nativeness is table stakes, not a differentiator.** ecu.test, RemotiveLabs, and Vector all have a CI story. The differentiator is the **delivery model**: Apache-2.0 core, no dongle, no separate SKU for headless use, transparent pricing.

1.5 What the regulatory research obliges L0–L3 to get right

India's **AIS-189** (drafted by ARAI's own standards committee; new vehicle types from **Oct 2025**, all types by **Oct 2028**) names its expected test methodology explicitly: "*fuzz testing of UDS, DoIP and SOME/IP.*"

That is an L4 activity — but it constrains L0–L3 now, because L4 is built *on top of* this layer:

- **UDS and DoIP must be first-class and rock-solid** (DIAG-02/03/04). They are named in a regulation our beachhead market must comply with.
- **DIAG-05's interception API must survive being driven by an external fuzzer**, including across a process boundary — because **boofuzz** (GPL-2.0) and CaringCaribou (GPL-3.0) cannot be linked into a proprietary L4 (C-9 reasoning applies to GPL as well as LGPL).
- **SOME/IP is named too.** It is *not* in v0.1 scope, and that is a considered choice — but the plan should stop describing it as merely "out of scope" and record it as **the first fast-follow after v0.1**, because a regulation our target customers must satisfy names it directly.

2. The Agentic Development Model

2.1 The unit of work is the loop

A **loop** is a work unit defined such that an agent can iterate autonomously — implement, run the verifier, read the failure, revise — terminating on a machine-checkable condition.

In Tapwright's terms, a loop is a `PROCESS.md` issue, with five things pinned down before implementation starts:

1. **Goal** — one sentence, one capability.
2. **Oracle** — the executable authority that decides correctness. *Not* "tests the agent will write." A pre-existing, independently-authored source of truth. **This is the field `PROCESS.md`'s test-plan step does not currently require, and the most important addition this plan makes.**
3. **Exit criteria** — the machine-checkable condition that ends the loop.
4. **Blast radius** — the explicit file set the agent may modify.
5. **Escalation trigger** — when the agent must stop and hand back.

If you cannot write the oracle, **you do not have a loop** — you have a research task that needs a human (§2.4).

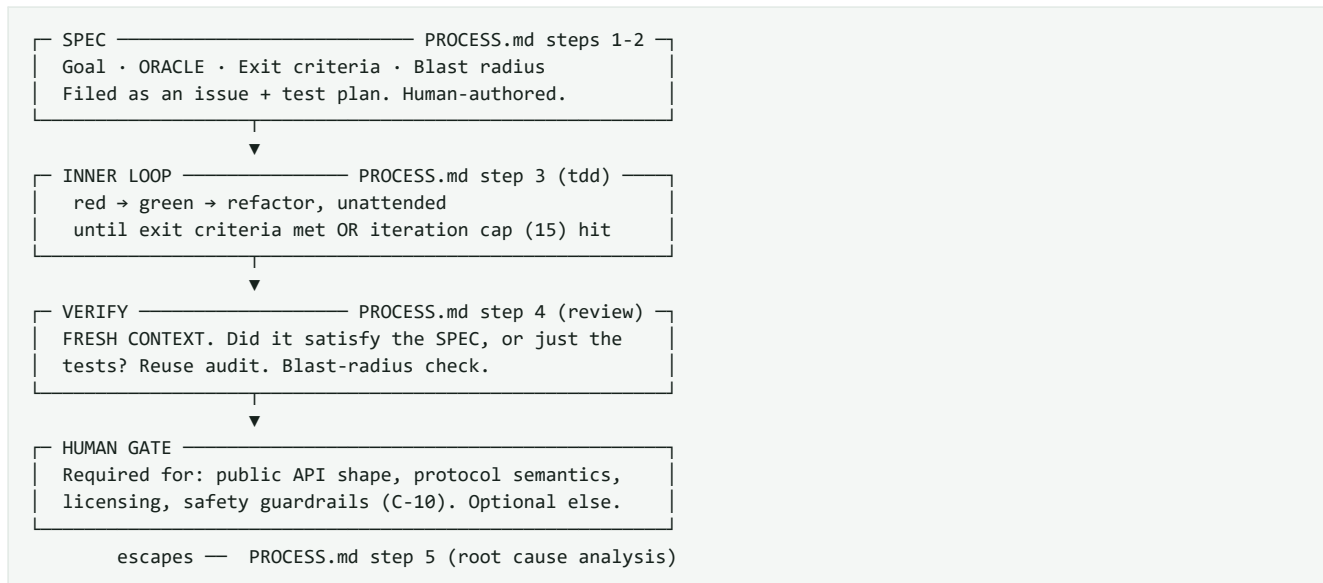
2.2 Why this codebase suits the method

Most projects struggle with agentic development because the oracle problem is hard. Tapwright is a rare case where oracles are abundant:

Oracle source	Why it exists here	Loops served
Published ISO specifications	UDS (14229), DoIP (13400), ISO-TP (15765-2) are precisely specified with deterministic request/response semantics	DIAG-01 ... DIAG-07
Reference implementations	We wrap <code>udsoncan</code> , <code>cantools</code> , <code>python-can</code> — differential testing against the library directly is always available	HAL-, BUS-, DIAG-*
Hermetic, free test environment	<code>vcan</code> gives a real kernel CAN bus with no hardware; tests run identically on a laptop and in CI	Every loop
The product is a test framework	Tapwright is pytest-based, so harness and product share a substrate	RUN-*
Recorded traces	BLF/ASC/MDF4 files are byte-exact fixtures — decode either matches or it doesn't	BUS-05, BUS-06

The honest corollary: **where the oracle is weak, the loop must be human-gated**. Three areas here have weak oracles and are marked as such: ODX semantic correctness (DIAG-06), real-hardware timing (HAL-03/04/05/06), and developer experience (RUN-09).

2.3 Loop anatomy, mapped to `PROCESS.md`



The verify stage must not be the same agent session that wrote the code. An agent that spent 40 iterations converging is the worst possible judge of whether the result matches the original intent. `PROCESS.md` already requires one reviewer approval; this plan adds the constraint that the reviewer did not supervise that loop.

2.4 When a loop cannot be closed

1. **Build the oracle first, as its own loop.** Most "unloopable" work is two loops. INF-04 and INF-05 exist precisely for this.
2. **Narrow the goal until an oracle exists.** "Import ODX" has no clean oracle. "Given this PDX file, resolve DID `0xF190` to VIN" does.
3. **Mark it human-led.** API ergonomics, naming, interface shape — taste is the requirement. Not a failure of the method; the method working.

2.5 Loop taxonomy

Type	Description	Typical oracle	Autonomy
W — Wrap	Expose an existing OSS library through our API	Differential test vs. the library called directly	High
P — Protocol	Implement protocol behaviour per spec	Conformance fixtures + simulated ECU	High
I — Integration	Make two of our own components work together	End-to-end test on vcan	High
H — Harden	Error paths, edge cases, property/fuzz	Hypothesis strategies; invariants	High
X — Infrastructure	Repo, CI, tooling, fixtures	The pipeline runs green	Medium
D — Design	Public API shape, ergonomics, docs	Human judgement	Low — human-led

Roughly 80% of the backlog is W/P/I/H. That ratio is the practical argument for the method on this specific project.

3. The Verification Ladder

Every loop declares the highest tier it must reach. Higher tiers subsume lower ones.

Tier	Gate	Mechanism	Repo status
T0	Static	<code>ruff check + ruff format --check + mypy</code> + SPDX header check + licence scan	● <code>ruff</code> done; <code>mypy</code> , <code>SPDX</code> , licence scan missing
T1	Unit	<code>pytest</code> unit tests, no I/O	● <code>pytest</code> runs; no <code>tests/unit/</code> tier
T2	Integration	Full stack against <code>vcan</code> + virtual ECU, zero hardware	● <code>TODO(M1)</code> in <code>ci.yml</code>
T3	Differential	Our output vs. the wrapped library called directly, or vs. a recorded golden trace	● Missing
T4	Property/fuzz	<code>hypothesis</code> -generated inputs; invariants hold; no crash on malformed input	● Missing
T5	Human	Reviewer approval against the SPEC, not against the diff	● One reviewer required per <code>PROCESS.md</code>

T3 is the tier that makes this project work. Because Tapwright wraps rather than rewrites, we can almost always ask: *does our abstraction produce byte-identical results to calling the underlying library directly?* That is a strong, cheap, agent-runnable oracle catching the most dangerous failure mode in this domain — a **silently wrong decode**, which produces plausible numbers with no error raised.

3.1 Mandatory tiers by layer

Layer	Minimum tier	Rationale
L0 (<code>hal</code>)	T3	Wrong frames are silently wrong; differential vs. <code>python-can</code> required
L1 (<code>buses</code> , <code>dbc_arxml</code> , <code>trace</code>)	T4	Decode errors are the highest-consequence bug class in the product
L2 (<code>diag</code>)	T4	Protocol state machines have deep edge cases; malformed-response handling is safety-adjacent
L3 (<code>runner</code> , <code>report</code>)	T2	Failures here are loud — a test that doesn't run is obvious
INF	T1	The pipeline itself is the verifier

3.2 Coverage gate

Per `DECISIONS-RECORD.md` §4: **85% line + 75% branch on L0-L2, as a ratchet** (may rise, never fall). No coverage gate on L3 beyond T2. CI already runs `pytest --cov=tapwright`; the ratchet logic is loop INF-02.

In `AGENTS.md`, verbatim: *"Coverage is not a goal. Assertions are. A test that executes code without asserting on its behaviour will be rejected in review regardless of its effect on coverage."*

4. Loop Substrate — Repo, Harness, CI

4.1 Repository layout

Existing structure ●, with additions this plan requires ●:

tapwright/	
└─ src/tapwright/	
└─ hal/	● L0 – backends, capability model
└─ buses/	● L1 – restbus, cyclic send
└─ dbc_arxml/	● L1 – database load/decode
└─ trace/	● L1 – BLF/ASC/MDF4 I/O
└─ diag/	● L2 – UDS, DoIP, ISO-TP, ODX, SOVD
└─ runner/	● L3 – pytest plugin, YAML collection, CLI
└─ report/	● L3 – HTML/JSON/ATX output
└─ tests/	
└─ unit/	● T1
└─ integration/	● T2 – vcan required
└─ differential/	● T3 – vs. wrapped libraries
└─ property/	● T4 – hypothesis
└─ fixtures/	● THE ORACLE LIBRARY (§4.3)
└─ databases/	golden DBC / ARXML / LDF / A2L
└─ traces/	recorded BLF / ASC / MDF4
└─ expected/	known-good decode outputs
└─ odx/	sample PDX packages
└─ tools/virtual_ecu/	● the simulated UDS/DoIP ECU (§4.4)
└─ docs/	● requirement chain (needs licence-table sync)
└─ AGENTS.md	● the agent contract (§4.2)
└─ LOOPS.md	● live loop backlog + status
└─ CODEOWNERS	● protects fixtures/ (§4.5)
└─ PROCESS.md	● the per-change workflow
└─ .github/workflows/	● ci.yml exists; needs T2-T4 jobs + guardrails

4.2 AGENTS.md — the agent contract

The repo has `.claude/skills/` (per-step interactive guidance) but **no repo-root agent contract**. These are complementary: skills describe *how to do a step*; `AGENTS.md` states *invariants that hold regardless of which step you are in*. It must stay short enough to be reliably followed.

Required contents:

- 1. Invariants at every commit** — tests green, lint clean, SPDX headers present, DCO sign-off, Conventional Commit message, no new dependency without a licence entry, **no L4/L5 code in this repo**.
- 2. The reuse rule, bluntly** — *"If `python-can`, `cantools`, `udsoncan`, `doipclient`, or `asammdf` already does this, wrap it. Do not reimplement it. A PR that reimplements wrapped-library functionality will be rejected regardless of whether tests pass."*
- 3. The oracle rule** — *"Do not modify files under `fixtures/` or `tests/differential/expected/` to make a test pass. If a fixture appears wrong, stop and escalate."*
- 4. The forbidden list** — C-10 (no seed-to-key derivation, no key databases, no bypass helpers); no vendoring of LGPL code (C-9); no network calls in tests.
- 5. The coverage rule** — verbatim text from §3.2.
- 6. Escalation protocol** — when to stop and ask, and what to include.

Item 3 deserves emphasis: **the most likely way agentic development fails on Tapwright is an agent "fixing" a failing differential test by editing the expected output**. That converts a caught bug into a permanent silent defect.

4.3 The fixture corpus is the most valuable artifact in the plan

`fixtures/` is not test scaffolding; it is the accumulated oracle that makes every subsequent loop cheaper and safer. Investment compounds. Build early (INF-04), grow deliberately.

Fixture class	Contents	Provenance requirement
Databases	DBC, ARXML, LDF, A2L covering multiplexing, extended IDs, scaling edge cases, negative offsets	Publicly-licensed or self-authored; provenance recorded per file
Traces	BLF, ASC, MDF4 recordings with known content	Self-generated on <code>vcan</code> where possible, so licensing is unambiguous
Expected outputs	Decode results as JSON	Human-verified at creation. Never regenerated to match code.
ODX/PDX	Sample diagnostic descriptions	Openly-licensed samples only — no OEM-proprietary data in a public repo, ever

The provenance requirement is not bureaucratic. Shipping an OEM-proprietary DBC or PDX in a public Apache-2.0 repository would be a serious legal problem, and it is exactly the kind of thing an agent might do while "finding a realistic test fixture."

4.4 The virtual ECU (INF-05) is load-bearing three times over

Per `ADR-005`, the `vcan`-based simulated ECU is simultaneously the **onboarding demo**, the **CI fixture**, and **every loop's integration oracle**.

It therefore cannot be a throwaway script. It needs its own tests, a documented scenario format, and **configurable failure injection** (negative response codes, timeouts, malformed frames) — because T4 hardening loops need an ECU that misbehaves on demand.

Per `DECISIONS-RECORD.md` §3, it is **`vcan`-only** — **no Renode in v0.1** — but its scenario interface must be designed so a Renode-backed implementation can be swapped in later without changing test code.

4.5 CI as the loop runtime

Current `ci.yml` has `lint` and `test`. Target shape:

```
jobs:
  t0-static:      ruff check · ruff format --check · mypy · spdx-check · licence-scan
  t1-unit:        pytest tests/unit
  t2-integration: modprobe vcan; ip link add vcan0; pytest tests/integration
  t3-differential: pytest tests/differential
  t4-property:    pytest tests/property --hypothesis-profile=ci
  coverage-ratchet: fail if line <85% or branch <75% on L0-L2; raise stored floor on improvement
  guardrails:     fixture-immutability · forbidden-symbol-scan (C-10) · blast-radius · LGPL-vendoring
```

The **guardrails job** is the **agentic-specific addition and is non-optional**. It fails the build if:

- any file under `fixtures/` or `tests/differential/expected/` changed without a `fixture-change`: commit trailer and CODEOWNERS approval;
- any forbidden symbol appears (seed-to-key derivation patterns, key tables — C-10);
- LGPL source has been vendored into the tree (C-9);
- files changed outside the loop's declared blast radius.

5. Loop Backlog

37 loops. **Must** = required for v0.1. **Should** = target, droppable under pressure. **Could** = fast-follow.

Type: W/P/I/H/X/D per §2.5 · *Tier*: highest required verification tier · *It.*: estimated agent iterations (planning signal, not a commitment) · *Status*: against the repo as of 2026-08-14.

5.1 INF — Infrastructure (the substrate; must land first)

ID	Goal	Type	Tier	Oracle	It.	Status
INF-01	Repo skeleton: pyproject , Apache-2.0, SPDX headers, CONTRIBUTING, CoC, DCO	X	T0	Pipeline green; SPDX check passes on all files	2	● Done — only SPDX header enforcement outstanding
INF-02	CI: T0–T2 jobs incl. vcan bring-up + coverage ratchet	X	T2	A trivial vcan test passes in CI; ratchet blocks a coverage drop	4	● Partial — resolves the TODO(M1) in ci.yml
INF-03	Automated licence gate + sync corrected licence table into docs/	X	T1	Adding a GPL test-dependency fails the build; docs/ matches FW-REQ-010/012/019	3	● Must
INF-04	Fixture corpus scaffolding + provenance manifest format	X	T1	Manifest validates; every fixture has recorded provenance	3	● Must
INF-05	Virtual UDS/DoIP ECU on vcan , scenario-configurable, failure injection. Evaluate reuse before building — see note	P	T3	Responds correctly to a udsoncan client used directly	8	● Must — start first
INF-06	AGENTS.md + CODEOWNERS + blast-radius config	D	T0	Human-authored; reviewed	1	● Must
INF-07	Loop telemetry: iterations-to-green, human-touch, escapes	X	T1	LOOPS.md auto-updates from CI metadata	3	● Should
INF-08	Docs site + executable examples (doctest in CI). Seed from the existing knowledge-base/05-training-labs/ sequence	X	T1	Every code sample in docs runs in CI	4	● Must

INF-05 is the highest-leverage loop in the entire plan. Every T2/T3 gate depends on it. Under-investing here caps the autonomy of all 29 downstream loops. Budget generously.

INF-05 — apply C-4 to the test harness, not just the product. The plan's central discipline is *reuse, don't rewrite*. That discipline applies to our own infrastructure too, and the research already found a candidate: [lbenthins/ecu-simulator](#) — an OSS tool that "simulates vehicle diagnostic services... to test tools that support UDS (ISO 14229) and ISO-TP (ISO 15765-2)." That is a large fraction of INF-05's job, already written. **Before writing a line of simulator code, spend half a day on:** (a) does it cover our scenario needs? (b) what licence? (c) does it support the *failure injection* T4 hardening loops require — NRCs, timeouts, malformed frames — or only happy-path responses? Wrapping it, forking it, or taking it purely as a reference are all better outcomes than reinventing it by default. Also worth reading first: **RemotiveLabs'** [RemotiveBus](#), a Docker network plugin doing SocketCAN-to-container bridging and inter-container [vcan](#) — the same problem RUN-08 will hit, already solved by someone shipping to Volvo.

5.2 HAL — L0 Hardware Abstraction ([src/tapwright/hal/](#))

ID	Goal	Type	Tier	Oracle	It.	Pri
HAL-01	Core Bus interface + capability model; config-driven backend selection	D+W	T3	Same test passes unchanged across ≥ 2 backends	6	Must
HAL-02	SocketCAN backend incl. vcan	W	T3	Differential vs. python-can directly; candump cross-check	4	Must
HAL-03	gs_usb backend (CANable 2.0 class)	W	T3	Differential vs. python-can ; manual hardware sign-off	5	Must
HAL-04	Kvaser canlib backend	W	T3	Differential vs. python-can ; manual hardware sign-off	5	Must
HAL-05	PEAK PCANBasic backend	W	T3	As above	4	Should
HAL-06	Vector XL backend	W	T3	As above	5	Should
HAL-07	Capability detection + graceful degradation (CAN-FD op on classic-CAN device $\rightarrow$ clear error, never silent failure)	H	T4	Property test: no silent capability mismatch across backend/op matrix	5	Must
HAL-08	LGPL isolation for python-can — dependency only, never vendored (C-9)	X	T1	Build fails if python-can source is vendored into the tree	2	Must

Human-gated: HAL-03/04/05/06 each need a physical-hardware sign-off no agent can perform. Loops close at T3-on-[vcan](#); a human runs the same suite against real hardware and records the result. **This is the main non-parallelisable dependency in the plan** — schedule it deliberately.

5.3 BUS — L1 Bus & Measurement Core ([buses/](#), [dbc_arxml/](#), [trace/](#))

ID	Goal	Type	Tier	Oracle	It.	Pri
BUS-01	DBC load + decode/encode via cantools	W	T4	Differential vs. cantools direct + golden expected outputs	5	Must
BUS-02	ARXML load + decode; log upstream gaps rather than forking. Adopt a dual-specification path — see note	W	T4	Differential vs. cantools ; known-gap list documented	7	Must
BUS-03	LDF (LIN) database support	W	T3	Differential vs. cantools	4	Should
BUS-04	A2L parse (read-only; no calibration write)	W	T3	Parse golden A2L; assert no write API is exposed	4	Should
BUS-05	Trace I/O: BLF + ASC read/write	W	T4	Round-trip byte-fidelity on recorded fixtures	6	Must
BUS-06	MDF4 via asammdf , optional extra + LGPL isolation (C-9)	W	T3	Core installs and passes without the extra; extra round-trips MDF4	5	Should
BUS-07	Restbus / cyclic-send engine, multi-node, DBC-driven cycle times	P	T4	Timing within tolerance over N seconds on vcan ; no drift	8	Must
BUS-08	Signal-level subscribe/filter API over live traffic	I	T2	Named-signal subscription yields decoded values from live vcan	5	Should
BUS-09	Ethernet restbus basics	P	T2	Frames observed on a virtual interface	6	Could

BUS-02 — borrow RemotiveLabs' "dual specification strategy." Their RemotiveTopology lets a team start with a **lightweight, hand-written topology description** and only graduate to full ARXML as the project matures. That directly addresses the real adoption barrier: ARXML is painful to author, and demanding a complete one on day one excludes exactly the small EV-OEM and Tier-2 teams in our beachhead. Concretely: accept a simple YAML/DBC-level description **as a first-class input**, with ARXML as the richer option — rather than treating ARXML as the only "proper" path. This costs little at design time and is expensive to retrofit.

5.4 DIAG — L2 Diagnostics Engine ([src/tapwright/diag/](#))

ID	Goal	Type	Tier	Oracle	It.	Pri
DIAG-01	ISO-TP transport via can-isotp (MIT — verified)	W	T3	Multi-frame segmentation/reassembly vs. library direct	5	Must
DIAG-02	UDS client core via udsoncan : session control, DTC read/clear, RDBI, routine control	W	T4	Differential vs. udsoncan direct against INF-05	8	Must
DIAG-03	DoIP transport via doipclient + entity discovery	W	T3	Differential vs. doipclient direct	6	Must
DIAG-04	Transport-agnostic connection abstraction — identical UDS API over CAN and Ethernet; shaped so SOVD can slot in later (DECISIONS-RECORD.md §7)	I	T3	The <i>same</i> test body passes over both transports, unmodified	6	Must
DIAG-05	Interception/observer hooks — externally-wrappable API (ADR-004), must work across a process boundary	D+I	T2	A third-party wrapper can observe and modify a request/response without forking, from a separate process	5	Must
DIAG-06	ODX/PDX read-only import → DID/routine name resolution	W	T3	Named resolution on golden PDX; human-verified semantics	8	Should
DIAG-07	SOVD client (REST/JSON, ISO 17978)	P	T3	Against a mock SOVD endpoint; self-description parsed correctly	8	Should
DIAG-08	C-10 guardrail : 0x27 mechanics only; CI scan blocks key-derivation code	H	T1	Introducing a seed-to-key routine fails the build	3	Must
DIAG-09	Malformed-response hardening: NRCs, timeouts, truncated/oversized frames	H	T4	Fuzzed responses from INF-05; no crash, no hang, clear errors	7	Must

On DIAG-05 and the L4 licensing constraint: these hooks were specified so a future Gallia-based L4 could wrap L2. The L4 licensing finding makes them more important — **boofuzz is GPL-2.0** and **CaringCaribou is GPL-3.0**, so the paid L4 layer must invoke them as *separate processes* rather than link them. A process-boundary-friendly interception API at L2 is what makes that architecture legal.

On DIAG-06 (weak oracle): ODX semantic correctness cannot be fully machine-verified. The loop closes on structural correctness; semantic spot-checks are a T5 human gate.

5.5 RUN — L3 Test Authoring & CI Runner ([runner/](#), [report/](#))

ID	Goal	Type	Tier	Oracle	It.	Pri
RUN-01	pytest plugin: bus , uds_client , virtual_ecu fixtures	D+I	T2	A user test using only fixtures passes with no boilerplate	6	Must
RUN-02	Declarative YAML test format → pytest collection	P	T3	YAML suite and equivalent Python suite produce identical results	8	Should
RUN-03	HTML report	W	T2	Report renders; contains every result; deterministic output	4	Must
RUN-04	JSON / ATX-style machine-readable report (TOOL-REQ-032)	W	T2	Schema-validates; round-trips	4	Must
RUN-05	Unified CLI — single entry point for all three invocation modes (ADR-001)	D+I	T2	Identical command works on laptop, headless bench, and container	5	Must
RUN-06	GitHub Actions example + reusable composite action. Benchmark against vectorgrp/ci-siltest-demo	X	T2	Example repo goes green from a cold clone, with fewer required steps than the Vector demo	4	Must
RUN-07	GitLab CI example	X	T2	As above	3	Should

ID	Goal	Type	Tier	Oracle	It.	Pri
RUN-08	Container image published alongside PyPI package (FW-REQ-021). Study RemotiveBus for container↔vcan networking	X	T2	<code>docker run</code> executes the quickstart with no host setup	4	Must
RUN-09	Time-to-first-green-test optimisation (NFR-005)	D	T5	New user reaches a passing test in < 1 hour from <code>pip install</code> — measured on real people	4	Must

RUN-09 is deliberately human-led. Its oracle is a stopwatch and a person who has never seen the tool. Agents can prepare candidate quickstarts; only observed human trials close it. Run it at least twice with different subjects.

RUN-06 has a concrete benchmark, which is unusual and valuable. Vector publishes `vectorgrp/ci-siltest-demo` — their own public reference implementation of automated SIL testing in CI. Clone it, read its workflow file, and count the steps a user must perform before a first green run. **Our example must require materially fewer, and no proprietary toolchain.** If we cannot beat the incumbent's own public demo on setup friction, the CI-native positioning does not hold up.

RUN-09 and INF-08 already have a first draft written. `knowledge-base/05-training-labs/` contains a nine-lab sequence (environment setup → raw CAN → DBC decode → UDS session → DoIP → pytest CI test → restbus → capstone pipeline) that maps almost one-to-one onto L0-L3, and Lab 08 is a miniature of the finished product. Use it as the docs backbone and the RUN-09 onboarding script rather than starting from a blank page — it also doubles as onboarding for new engineers joining the team.

6. Schedule — M1 to M6 (26 weeks)

Milestone	Weeks	Loops	Exit criterion
M1 — Substrate	W0-3	INF-02...06 (INF-01 • done), HAL-01, HAL-02, DIAG-01 (thin), RUN-01 (thin)	One end-to-end loop runs unattended: an agent implements a UDS request against INF-05 and closes at T3 with no human in the inner cycle
M2 — Core diagnostics	W3-8	HAL-03, HAL-04, HAL-07, HAL-08, BUS-01, DIAG-02, DIAG-03, DIAG-04, DIAG-05	Same UDS test passes unmodified over CAN and DoIP; DBC decode differential-clean; ≥2 hardware backends signed off
M3 — Trace & reporting	W8-12	BUS-02, BUS-05, BUS-06, BUS-07, RUN-03, RUN-04	Restbus runs multi-node; BLF/ASC round-trip byte-exact; HTML + JSON reports emitted
M4 — The CI story	W12-16	RUN-05, RUN-06, RUN-07, RUN-08, INF-07, INF-08	Cold-clone example repo goes green in GitHub Actions with zero hardware; docs site live (FW-REQ-066)
M5 — ODX, SOVD & breadth	W16-22	DIAG-06, DIAG-07, DIAG-09, BUS-03, BUS-04, BUS-08, HAL-05, HAL-06, RUN-02	ODX names resolve on a golden PDX; SOVD client works against a mock; hardening suites green
M6 — Pilot & harden	W20-26	BUS-09, RUN-09, real-hardware sign-offs, escape-defect burn-down	Design partner runs Tapwright on their own bench; RUN-09 measured under 1 hour

Design-partner targeting is no longer generic. The research names both the institutions and the opening question:

- **ARAI** (Pune) and **ICAT** (Manesar) are India's government-designated agencies for **both CSMS certification and vehicle type approval** — they are the gatekeepers, not merely labs. ARAI sits in Pune, the same city as India's densest automotive-software engineering base.
- The **AIS-189** methodology they administer explicitly expects *"fuzz testing of UDS, DoIP and SOME/IP using **CANoe** + **boofuzz**."* That is a specific, expensive, stitched-together workflow — and a genuinely well-informed conversation opener: *"How does your team actually run the **CANoe** + **boofuzz** AIS-189 workflow day to day?"*
- Per `market-apps-documentation/ais-189-15-evidence-requirements.md`, **AIS-189's evidence submission format is negotiated with ARAI/ICAT rather than published.** A partner who walks us through one real submission hands us knowledge

that cannot be obtained by reading the standard — the most valuable single asset the future L5 could have. This reframes M6's design-partner search from "find a friendly first user" to "find the partner who unlocks L5."

M5 and M6 overlap by design (W20–22).

6.1 Critical path

```
INF-02 → INF-05 ┤→ HAL-01 → HAL-02 → DIAG-01 → DIAG-02 → DIAG-04 → RUN-01 → RUN-05 → RUN-08
                  ↳ BUS-01 → BUS-07
```

INF-05 (the virtual ECU) gates almost everything. It is the first loop that can genuinely slip the whole schedule. Treat any delay there as a project-level risk.

Because INF-01 is already complete, **M1 starts on INF-02/INF-05 immediately** rather than on repo scaffolding — roughly a week of head start against the original plan.

6.2 Parallelisation

After M1, three streams run concurrently with low coupling: **HAL/BUS** (bus-level), **DIAG** (protocol), **RUN/INF** (developer experience). This maps to the 3–4 engineer Phase 1 headcount, each owning a stream and supervising its loops rather than hand-writing all its code.

7. Team & Roles Under an Agentic Model

Role	Responsibility	Notes
Loop Author	Writes SPECS: goal, oracle, exit criteria, blast radius. Owns fixture quality	The highest-value human work in this plan. A good SPEC is worth more than a good implementation, because the implementation is cheap and the SPEC is what makes it correct
Loop Supervisor	Runs loops, triages escalations, decides when to narrow a goal or take it human-led	Typically the same person as Loop Author, per stream
Independent Verifier	Reviews closed loops against the SPEC with fresh context; runs the reuse audit	Must not be the person who supervised that loop. Rotate across streams
Integrator / Maintainer	Owns <code>main</code> , CODEOWNERS on <code>fixtures/</code> , dependency and licence decisions, releases	Also the human gate for C-9 and C-10

Two rules worth stating explicitly:

- 1. Nobody verifies their own stream.** The cheapest defence against oracle capture.
- 2. Physical-hardware sign-off is a named human task with a named owner**, scheduled into M2 and M5. It cannot be automated and will not happen by accident.

8. Metrics

Instrument from M1 (INF-07) — the method is being validated alongside the product.

Metric	Definition	Target	Read as
Iterations-to-green	Median agent iterations per closed loop, by type	W/P/I ≤ 8	Rising = SPECS or oracles degrading

Metric	Definition	Target	Read as
Human-touch rate	% of loops needing unplanned human intervention	< 30% for W/P/I/H	High = goals too broad, or oracle too weak
Escape rate	Defects found <i>after</i> a loop closed, per closed loop	< 0.2	The key quality metric. Rising = verification ladder too shallow
Oracle coverage	% of loops reaching their declared tier	100%	Any gap is a silent quality risk
Fixture-tamper attempts	Guardrail blocks on fixture edits	Track, don't target	Non-zero is expected; a spike means AGENTS.md needs strengthening
Blast-radius violations	Files touched outside declared scope	→ 0	Persistent violations mean scopes are wrong, not that the rule is wrong
Time-to-first-green-test	Cold <code>pip install</code> → passing test (real users)	< 1 hour	The product's own adoption metric (NFR-005)

Escape rate matters most. Everything else measures efficiency; escape rate measures whether the method produces correct software. If iterations-to-green is excellent and escape rate is climbing, loops are being gamed and the ladder needs to go deeper — not faster.

PROCESS.md step 5 (root cause analysis) is the existing mechanism for handling escapes; INF-07 adds the counting.

9. Risk Register

9.1 Risks specific to agentic development

Risk	Sev	Mitigation
Oracle capture — agent edits a fixture or expected output to make a failing test pass, converting a caught bug into a permanent silent defect	HIGH	CODEOWNERS on fixtures/ ; guardrail CI job; explicit prohibition in AGENTS.md ; fixture edits require a commit trailer + human approval
Silently wrong decode — plausible numbers, no error, no test failure	HIGH	T3 differential mandatory on all decode paths; T4 property tests on L1/L2; golden outputs human-verified at creation
Reimplementation drift — agent rewrites what cantools/udsoncan already does, passing tests while destroying the reuse strategy	HIGH	Reuse audit standing item in independent verification; blunt rule in AGENTS.md ; PR template asks "what existing library does this?"
Spec drift — loops no longer satisfy the original requirement IDs	MED	Requirement IDs in test names (PROCESS.md already requires them in issues); traceability report in CI
Licence creep — a convenient GPL/AGPL dependency enters the Apache-2.0 core	MED	INF-03 automated gate; pyproject.toml 's deliberately-empty dependencies is a good starting posture
Proprietary fixture contamination — an OEM DBC/PDX lands in a public repo	MED	Provenance manifest mandatory (INF-04); review gate on all fixture additions
Context loss between loops	MED	ADR discipline; LOOPS.md as durable state
Over-trust in green CI	MED	Escape-rate tracking; periodic manual exploratory testing against real hardware

9.2 Technical & delivery risks

Risk	Sev	Mitigation
INF-05 slips — gates the entire critical path	HIGH	Start W0; most experienced engineer; accept a narrow first version (UDS-over-CAN only) and extend
Real-hardware behaviour diverges from <code>vcan</code> (timing, error frames, driver quirks)	HIGH	Named sign-off tasks in M2/M5; hardware ordered in W0, smoke-tested W1-2 (DECISIONS-RECORD.md §6)
<code>cantools</code> ARXML depth gaps block BUS-02	MED	Documented known-gap list; upstream contributions rather than a fork; narrow v0.1 ARXML claims honestly
LGPL isolation implemented incorrectly	MED	HAL-08 + INF-03; legal opinion before first public release (DECISIONS-RECORD.md §5)
ODX semantics wrong but structurally valid	MED	T5 human gate; ship read-only; explicit docs about coverage limits
SOVD spec access / early standard churn	LOW	Mock-endpoint development; remains <i>Should</i>
Scope creep from L4/L5 into this repo	MED	ADR-003/007 enforced by a CI check, not discipline alone

9.3 Positioning risks surfaced by the competitive scrapes

These do not threaten delivery; they threaten the *story* we tell about it, which matters just as much for a bottom-up adoption product.

Risk	Sev	What the research found	Response
"Zero-install trial" is a weaker claim than assumed	MED	Vector runs a browser-based CANoe Online Demo (KB0023426)	Stop leading with "no install." Lead with <i>free + scriptable + runs in your own pipeline + no dongle</i>
"CI-native" is table stakes, not a differentiator	MED	ecu.test ships <code>one:cx</code> , a named CI/CD platform; Vector documents Jenkins control and publishes <code>ci-siltest-demo</code> ; RemotiveLabs is CI-native with Volvo and Ford	Differentiate on delivery model — Apache-2.0 core, no dongle, no separate headless SKU — not on having CI
Free-tier success can still mean single-customer dependence	MED	EXAM is freeware with ~3,500 users — but ~2,500 of them are inside Volkswagen	A design partner is necessary but not sufficient. Track <i>distinct organisations</i> adopting, not just user count, against the 500-user Phase 1→2 gate
Feature-parity gaps are real and must be stated, not hidden	LOW	Vehicle Spy does full ODX read and write ; TSMaster ships free CCP/XCP calibration with write and FlexRay	Publish an honest capability matrix in the docs. Being narrower on purpose is defensible; appearing to have missed it is not

One finding that cuts the other way, and is worth morale: per

[market-apps-documentation/oss-dependency-health-comparison.md](#), our own dependencies are **healthier open-source projects than the incumbents' own OSS releases** — `cantools` (2,300) and `python-can` (1,600) against Vector's SIL Kit (209) and ETAS-backed openDuT (49). We are not betting on niche libraries while competitors ride thriving ecosystems; if anything it is the reverse.

10. Definition of Done — v0.1

Engineering release bar. Distinct from the business gate (≥500 engaged users + a paying design partner).

Functional

- [] All **Must** loops closed at their declared verification tier
- [] The same UDS test body runs unmodified over CAN (ISO-TP) and Ethernet (DoIP) — DIAG-04
- [] The same suite runs unmodified on a laptop, a headless bench, and a CI container — ADR-001 / RUN-05

- [] ≥3 hardware backends validated against physical devices by a human, results recorded
- [] Full test suite passes on `vcan` with zero physical hardware — C-8 / FW-REQ-031

Quality

- [] T0-T4 all green on `main`; escape rate < 0.2
- [] Coverage ratchet holding at ≥85% line / ≥75% branch on L0-L2
- [] Differential tests cover every decode path in L1 and L2
- [] DIAG-08 guardrail verified by a deliberate red-team commit that CI correctly rejects

Legal & licensing

- [] Apache-2.0 + SPDX headers on every source file
- [] Dependency licence manifest complete and CI-enforced; C-9 isolation verified
- [] `docs/framework-requirements.md` synced with the corrected licence table (FW-REQ-010/012/019)
- [] Every fixture carries recorded provenance; no OEM-proprietary data present
- [] **LGPL legal opinion obtained** — the one remaining external dependency

Distribution & DX

- [] `pip install tapwright` from PyPI; container image published (FW-REQ-020/021)
- [] Docs site live with executable, CI-tested examples (FW-REQ-066)
- [] **Time-to-first-green-test measured under 1 hour with ≥2 real users** (RUN-09)

Process

- [] Every closed loop has a recorded SPEC, oracle, and verification outcome
- [] Metrics dashboard live since M1
- [] `PROCESS.md` updated to incorporate the oracle requirement and fixture-immutability rule

11. Decisions

All eight open decisions were closed on 2026-08-14. Full rationale and the obligations each creates are in `DECISIONS-RECORD.md`; the discussion material is in `OPEN-DECISIONS-DISCUSSION-BRIEF.md`.

#	Outcome
1	DCO adopted (ADR-006 → <i>Accepted</i>) — already implemented in <code>CONTRIBUTING.md</code> . Plus new ADR-007 : the open/commercial split is a <i>repository</i> boundary, not a feature boundary
2	Name : <code>tapwright</code> . Repo <code>SKIFIN-IT-SERVICES/tapwright</code> live; PyPI name verified available 2026-08-14 — claim it
3	vcan-only for INF-05; interface must stay swappable so Renode can back it later
4	85% line / 75% branch on L0-L2, as a ratchet ; no coverage gate on L3
5	PyPI + container proceed; FW-REQ-022 single static binary BLOCKED ; licence table corrected; legal opinion still required before public release
6	~\$530 hardware ordered in W0 ; smoke test W1-2
7	SOVD stays Should ; DIAG-04 shaped so it slots in later
8	Plugin SDK stability promise deferred to v1.0 ; boundaries designed as public; DIAG-05 must work across a process boundary

The only remaining external dependency in the entire plan is the LGPL legal opinion, which blocks first public release but not development.

11.1 Name artifacts (resolved)

Artifact	Value
GitHub org / repo	SKIFIN-IT-SERVICES/tapwright
PyPI package	tapwright — available as of 2026-08-14, not yet claimed
Python import path	tapwright
Container image	ghcr.io/skifin-it-services/tapwright (suggested — GHCR aligns with the existing GitHub org)
Domain	not yet registered

Appendix A — Loop SPEC Template

Every loop starts as one of these. File as a GitHub issue per `PROCESS.md` step 1, with the test plan as step 2. Track status in `LOOPS.md`.

```
### <ID> — <one-line goal>

**Type:** W | P | I | H | X | D
**Layer:** L0 | L1 | L2 | L3 | INF      **Module:** tapwright.<module>
**Priority:** Must | Should | Could    **Required tier:** T0-T5

**Goal**
<One sentence. One capability. If it needs "and", split the loop.>

**Oracle**
<The executable authority. Name the exact file, fixture, or reference call.
  If this section is vague, the loop is not ready to start.>

**Exit criteria**
- [ ] <Machine-checkable condition>
- [ ] <Machine-checkable condition>

**Blast radius**
Allowed:  src/tapwright/<module>/**, tests/<tier>/<area>/**
Forbidden: fixtures/**, tests/differential/expected/**, anything outside the above

**Escalate to a human if**
- The oracle appears wrong
- Exit criteria cannot be met without touching forbidden paths
- Iteration cap (default 15) reached
- A new third-party dependency seems necessary

**Requirement traceability:** <TOOL-REQ-xxx / FW-REQ-xxx / ADR-xxx>
**Labels:** L0|L1|L2|L3 · must|should|could · enhancement
```

Appendix B — Requirement Traceability

Plan element	Satisfies
HAL-01...HAL-08	TOOL-REQ-001...009; FW-REQ-010/019; IV-HW-02/03/06
BUS-01...BUS-09	TOOL-REQ-014/015/017/021; FW-REQ-011/015/019
DIAG-01...DIAG-09	TOOL-REQ-022...027; FW-REQ-012/013/014/018; ADR-004; IV-STD-D1-15
RUN-01...RUN-09	TOOL-REQ-028...034; FW-REQ-016/020/021/030; ADR-001; NFR-005

Plan element	Satisfies
INF-01...INF-08	FW-REQ-017/031/040/041/042/050/052/060...066; ADR-003/005/007
§3 Verification ladder	FW-REQ-030/031/032; NFR-003
§9 Risk register	Extends 04- viability/recommendation.md with method-specific risks

Appendix C — Immediate Next Actions

Ordered. The first four are W0.

1. **Claim tapwright on PyPI** — verified available 2026-08-14; availability rots. Upload a `0.0.1.dev0` placeholder from the existing `pyproject.toml`.
2. **Order hardware** — 2× CANable 2.0, 1× Kvaser Leaf v3, cabling (~\$530). Assign a named owner.
3. **INF-06** — write `AGENTS.md` + CODEOWNERS. Cheap, and it constrains every loop that follows.
4. **INF-05** — start the virtual ECU. **First half-day: evaluate** `lbenthins/ecu-simulator` **for reuse** before writing anything. Longest pole; gates everything.
5. **INF-02** — resolve the `TODO(M1)` in `ci.yml`: `vcan` bring-up, T1–T4 job split, coverage ratchet, guardrails job.
6. **INF-03** — licence gate, and sync the corrected licence table into `docs/framework-requirements.md`.
7. **Amend PROCESS.md** — add the oracle requirement to step 2, and the fixture-immutability rule to step 3.
8. **Clone** `vectorgrp/ci-siltest-demo` and count its setup steps — that number is RUN-06's benchmark.
9. **Book the LGPL legal consult** — does not block development, but blocks public release; long lead time.
10. **Draft the ARAI/ICAT outreach** using the AIS-189 "CANoe + boofuzz" opener (§6). Long relationship lead time; starting in W0 costs nothing.

Appendix D — Research Findings Applied in v1.2

Traceability from the accumulated research to the specific plan changes it caused. Findings that confirmed existing decisions without changing them are omitted.

Finding	Source	Applied to
AIS-189 names " <i>fuzz testing of UDS, DoIP and SOME/IP</i> "; effective Oct 2025 / Oct 2028	arai-icat-india-labs.md	§1.5 — SOME/IP reclassified from "out of scope" to first fast-follow ; UDS/DoIP robustness reinforced as regulation-backed
ARAI/ICAT are gatekeepers for both CSMS certification and type approval; AIS-189 submission format is negotiated, not published	arai-icat-india-labs.md , ais-189-15-evidence-requirements.md	§6 — design-partner targeting made specific, with a concrete opener and the "unlocks L5" framing
<code>lbenthins/ecu-simulator</code> — OSS UDS + ISO-TP simulator already exists	knowledge-base/06-external-resources/10-github-hands-on-projects.md	INF-05 — mandatory reuse evaluation before building; C-4 extended to test infrastructure
<code>RemotiveBus</code> is a Docker network plugin doing SocketCAN bridging and inter-container <code>vcan</code>	remotive-labs.md	INF-05, RUN-08 — named as the reference implementation for container↔ <code>vcan</code> networking

Finding	Source	Applied to
RemotiveTopology's dual specification strategy — lightweight description first, ARXML later	remotivelabs.md	BUS-02 — lightweight input becomes a first-class path, not a fallback
vectorgrp/ci-siltest-demo is Vector's own public CI reference	10-github-hands-on-projects.md	RUN-06 — becomes a measurable benchmark (fewer setup steps than the incumbent's demo)
Vector runs a browser-based CANoe Online Demo	vector-canoe.md (KB0023426)	§1.4, §9.3 — "zero-install" claim downgraded; positioning rewritten
ecu.test ships one:cx ; RemotiveLabs is CI-native with Volvo/Ford	ecutest-tracetronic.md , remotivelabs.md	§1.4, §9.3 — CI-nativeness reclassified as table stakes, not differentiator
EXAM: ~3,500 users but ~2,500 inside VW alone	exam-micronova.md	§9.3 — Phase 1→2 gate should count <i>distinct organisations</i> , not just users
Vehicle Spy does ODX read and write; TSMaster ships free CCP/XCP write + FlexRay	vehicle-spy-intrepid.md , tsmaster-tosun.md	§1.4 — explicit, published capability-gap matrix rather than silent omission
boofuzz GPL-2.0, CaringCaribou GPL-3.0, Gallia Apache-2.0	14-security-tools-licensing-risk.md	§1.5, DIAG-05 — process-boundary requirement reinforced as a licensing necessity, not a nicety
Our deps are healthier OSS than incumbents' (cantools 2,300 vs SIL Kit 209)	oss-dependency-health-comparison.md	§9.3 — recorded as a counterweight to the incumbent-OSS risk narrative
SOVD is self-describing (works without an ODX file) but incumbents already ship it	sovd-standard-research.md	§1.4, DIAG-04/07 — kept as <i>Should</i> , but DIAG-04 must be shaped to accept it later
knowledge-base/05-training-1abs/ — 9 labs mapping 1:1 onto L0-L3	knowledge-base/	INF-08, RUN-09 — becomes the docs backbone, onboarding script, and RUN-09 trial material

Companion PDF: [DEVELOPMENT-PLAN-L0-L3-AGENTIC.pdf](#). Decisions: [DECISIONS-RECORD.md](#). Discussion material: [OPEN-DECISIONS-DISCUSSION-BRIEF.md](#). Source research: 01- through 07-, [market-apps-documentation/](#), [knowledge-base/](#).